

PulseRTL

AI-Assisted RTL Verification & Timing Analysis Platform

TECHNICAL ARCHITECTURE & WORKFLOW REPORT

Engineering Documentation Generated by Claude AI

Idea and designed by

SHREYAS YADAV M

B.Tech in VLSI Design and Technology

1. Executive Summary

PulseRTL is an AI-assisted RTL verification and timing analysis workflow platform designed for hardware engineers working with Verilog and SystemVerilog. The system orchestrates a structured, AI-driven pipeline that automates syntax checking, timing risk evaluation, testbench generation, waveform prediction, and engineering explainability across the full RTL analysis cycle.

The platform is positioned as a workflow-oriented engineering assistant, not a general-purpose AI chatbot. Its architecture integrates a Monaco-based code editor with a structured backend AI pipeline powered by the Gemini 2.5 Flash model, returning deterministic JSON-structured verification artifacts that drive a premium tabbed dashboard UI.

1.1 Platform Identity

Product Name	PulseRTL
Platform Type	AI-Orchestrated RTL Verification Workflow
Primary Language Support	Verilog, SystemVerilog (SV)
AI Backend	Gemini 2.5 Flash via direct REST API
Verification Pipeline Stages	8 Sequential Signoff Checkpoints
Output Format	Structured JSON — parsed into tabbed UI artifacts
Waveform System	AI-predicted SVG timing diagram with hexagon bus labels
HLS Support	C/C++, Python, VHDL → synthesizable RTL conversion

1.2 What PulseRTL Is — and Is Not

✓ PulseRTL IS	✗ PulseRTL IS NOT
- Workflow-oriented engineering assistant - AI-orchestrated verification pipeline - Frontend/backend AI engineering platform - Waveform prediction and visualization system - Engineering explainability interface	- A general-purpose AI chatbot - An LLM training or fine-tuning platform - A full RTL compiler or simulator - A production EDA tool (Quartus / Vivado) - A distributed cloud inference system

2. System Architecture

PulseRTL follows a minimal, layered architecture with a clean separation between the frontend workspace, the backend API layer, and the Gemini AI service. No distributed systems, agent clusters, or vector databases are present in the current implementation.

2.1 High-Level Architecture Overview



2.2 Core Architectural Boundaries

Frontend	React application hosted at / (Verifier) and /synthesizer (HLS Translator)
Code Editor	Monaco Editor — language-aware, supports Verilog / SystemVerilog / C / VHDL
API Handler	analysis.functions.ts — direct fetch() to Gemini REST endpoint
API Endpoint	generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent
Auth Model	GEMINI_API_KEY injected via environment variable (.env)
Response Format	Structured JSON parsed into AnalysisResult schema
State Manager	LocalStorage — transfers RTL code between /synthesizer and /
Waveform Renderer	WaveformViewer.tsx — SVG-based cycle-level timing diagram

2.3 Frontend Workspace Layout

The UI is organized as a full-window engineering dashboard, replacing the original chatbot-style landing page. The workspace is divided into three primary zones:

Zone	Contents
Top Navbar	Status indicators (GEMINI CONNECTED, WORKSPACE ACTIVE), layout toggles, navigation links
Left Panel	Monaco Editor with language selector, demo examples, RTL file upload controls
Right Panel	Engineering metrics scorecards + Agent Execution Pipeline (8 animated signoff stages)

Bottom Section	8-tab results panel: Issues, Timing Warnings, Optimized Code, Testbench, Edge Cases, Output Prediction, Explain Simply, Engineering Summary
----------------	---

3. AI Execution Pipeline

The verification pipeline executes 8 sequential signoff checkpoints in response to a submitted RTL file. Each stage is animated in real-time through the AgentPipeline component, driven by the response from the Gemini AI backend. The pipeline enforces a strict execution order — no stages are skipped, parallelized, or reordered.

3.1 Pipeline Execution Stages

#	Stage Label	Engineering Function
1	Parsing RTL	Tokenizes and parses the submitted Verilog/SystemVerilog source into an abstract syntax representation for downstream analysis.
2	Detecting Syntax Issues	Scans for syntax errors, undefined signal references, undeclared modules, and non-synthesizable constructs.
3	Evaluating Timing Risks	Identifies latch inference hazards, setup/hold violations, combinational loops, and asynchronous path risks in the design.
4	Optimizing Architecture	Proposes structural improvements: register balancing, pipeline stage insertion, and redundancy elimination for synthesis efficiency.
5	Generating Testbench	Produces a self-checking SystemVerilog testbench scaffold with clock generation, reset sequencing, and representative stimulus vectors.
6	Creating Edge Cases	Enumerates boundary conditions and corner-case input scenarios relevant to the detected circuit topology.
7	Predicting Outputs	Estimates signal behavior, transition sequences, and timing waveform patterns based on AI interpretation of the RTL logic.
8	Building Engineering Summary	Aggregates all findings into a structured summary: risk level, code quality score, timing stability index, and verification coverage estimate.

3.2 Gemini API Integration

The backend server function `analyzeCode()` dispatches a structured prompt to the Gemini 2.5 Flash endpoint via a `direct fetch()` call. The system instruction enforces a strict JSON schema response — no free-form prose is returned. Defensive parsing with fallback defaults ensures the UI always receives a valid `AnalysisResult` object.

Endpoint	<code>generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent</code>
Auth	<code>?key=\${GEMINI_API_KEY}</code> query parameter
Request Mode	Single-turn structured prompt with embedded RTL source
Response Schema	Strict JSON — <code>issues[]</code> , <code>timingWarnings[]</code> , <code>optimizedCode</code> , <code>testbench</code> , <code>edgeCases[]</code> , <code>outputPrediction</code> , <code>summary</code>
Error Handling	Fallback to default <code>AnalysisResult</code> with warning displayed in UI

Source File	src/lib/analysis.functions.ts
-------------	-------------------------------

4. Engineering Metrics Scorecards

Upon pipeline completion, four engineering metrics are computed from the structured JSON response and rendered as scorecard widgets in the right panel. These metrics provide an at-a-glance assessment of RTL quality without requiring the engineer to inspect detailed output tabs.

Metric	Field	Description
Risk Level	riskLevel	Percentage estimate of synthesis and verification risk. Displayed as High / Medium / Low with a progress gauge.
Code Quality Score	codeQualityScore	Cleanliness and structural integrity rating of the submitted RTL. Based on issue count and optimization opportunities.
Timing Stability	timingStability	Composite index derived from setup/hold hazard count, latch inference detections, and combinational path analysis.
Verification Coverage	verificationCoverage	Estimated testbench readiness percentage — reflects how well the generated testbench exercises the detected circuit logic.

4.1 Golden RTL Synthesis-Ready State

A conditional rendering path is triggered when the verifier detects high-quality RTL meeting both thresholds: a code quality score of 85% or above and zero detected syntax or structural issues. In this state, the comparative code editor view is suppressed and replaced with a synthesis-ready confirmation banner, eliminating visual noise for production-ready logic blocks.

Condition: Quality ≥ 85% AND Issues = 0 → Render: ✓ Hardware Implementation Synthesis Ready banner → Suppress: comparative code editor panels	Condition: Quality < 85% OR Issues > 0 → Render: side-by-side original vs optimized RTL editor → Highlight: syntax suggestions and optimization diffs
---	--

5. Verification Output Tabs

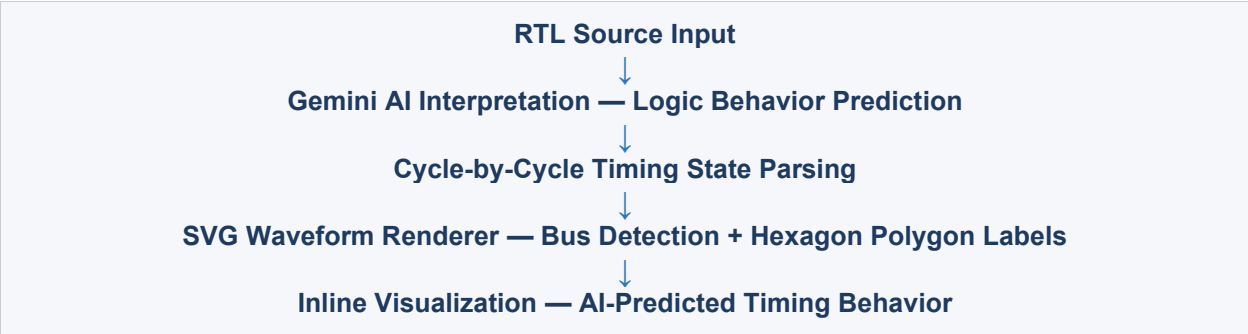
Analysis results are rendered into an 8-tab structured panel at the bottom of the workspace. Each tab targets a specific engineering concern, enabling engineers to navigate directly to relevant verification artifacts without scrolling through a monolithic output block.

Tab	Content & Engineering Purpose
Tab 1: Issues	Lists detected syntax errors, undefined references, undeclared modules, and non-synthesizable constructs with line-level context.
Tab 2: Timing Warnings	Focuses on latch inference, combinational loop detection, setup/hold hazard flags, and asynchronous path risks.
Tab 3: Optimized Code	Side-by-side or tabbed view of original RTL versus AI-suggested optimized RTL. Includes a Download Optimized RTL button (.sv file).
Tab 4: Testbench	Full self-checking SystemVerilog testbench with clock/reset scaffolding and representative stimulus. Download Testbench button included.
Tab 5: Edge Cases	Modular card list of recommended hardware test scenarios — boundary conditions and corner cases specific to the detected circuit topology.
Tab 6: Output Prediction	AI-predicted signal behavior, timing transition sequences, and waveform state descriptions based on RTL logic interpretation.
Tab 7: Explain Simply	Plain-language structural description of the RTL module's function — suitable for design reviews or cross-team explainability.
Tab 8: Engineering Summary	High-level executive briefing: risk level, issue count, timing risk count, code quality score, testbench status, optimization opportunity count.

6. Waveform Generation & Timing Diagram System

PulseRTL includes a dynamic SVG-based timing diagram emulator implemented in WaveformViewer.tsx. Rather than displaying static charts, the system generates waveform visualizations by parsing the AI-predicted output behavior cycle by cycle.

6.1 Waveform Generation Flow



6.2 Waveform Renderer Capabilities

Signal Types	Single-bit digital signals (clk, rst_n, valid, enable) and multi-bit buses (data_in, code_out)
Bus Rendering	Hexagon crossover transition polygons with embedded hex/decimal value labels
Clock Visualization	Consistent toggle intervals with proper H/L transitions
Timing Grid	Soft transparent vertical grid lines with T0, T1, T2... cycle markers
Label Centering	Labels auto-centered to the midpoint of extended signal states
Color System	Differentiated by signal type — clock (blue), reset (red), valid (green), buses (purple)
Implementation	WaveformViewer.tsx — pure SVG output, no external chart library dependencies

6.3 Example: 4-to-2 Priority Encoder Waveform Verification

The waveform system has been validated against a 4-to-2 priority encoder with valid output. The predicted timing behavior was verified against the expected encoder truth table:

data_in	valid	code_out	Verification
0000	0	xx / 00	✓ Reset / inactive
0001	1	00	✓ D0 active — lowest priority

0010	1	01	✓ D1 active
0100	1	10	✓ D2 active
1000	1	11	✓ D3 active — highest priority

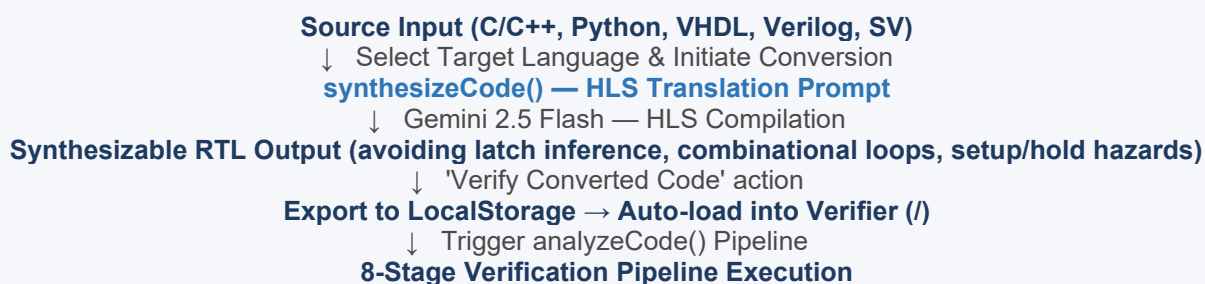
7. HLS Translation Workflow

The /synthesizer route provides an HLS (High-Level Synthesis) translation engine that converts higher-level or alternative HDL source code into synthesizable RTL. The `synthesizeCode()` backend function enforces translation constraints to minimize downstream verification issues.

7.1 Supported Input Languages

- C / C++ — behavioral algorithm descriptions
- Python — high-level logic specifications
- VHDL — legacy HDL conversion to SystemVerilog
- Verilog — dialect normalization to SV
- SystemVerilog — structural reorganization for synthesis

7.2 HLS Translation Flow



7.3 Dual-Engine Cohesion

The HLS translation prompt is calibrated to emit standard synthesizable digital blocks. Specifically, it is constrained to avoid: combinational loops, implicit latch inference, non-blocking/blocking assignment mixing in the same always block, and timing-sensitive path hazards. This calibration ensures that translated RTL passed to the verifier produces minimal structural warnings, enabling a reliable first-pass verification workflow.

8. Complete System Data Flow

The following table describes the end-to-end data flow through the PulseRTL platform, from RTL submission to engineering report generation.

Step	Origin		Destination	Payload
1	User / Monaco Editor	→	<code>analyzeCode()</code>	RTL source string + language tag
2	<code>analyzeCode()</code>	→	Gemini 2.5 Flash API	Structured prompt with RTL embedded
3	Gemini 2.5 Flash API	→	<code>analyzeCode()</code>	Raw JSON completion response
4	<code>analyzeCode()</code>	→	Frontend State	Parsed AnalysisResult object
5	AnalysisResult	→	AgentPipeline.tsx	Pipeline stage progression signals
6	AnalysisResult	→	ScoreCards.tsx	riskLevel, codeQuality, timingStability, coverage
7	AnalysisResult	→	OutputTabs.tsx	issues, warnings, code, testbench, edgeCases, summary
8	AnalysisResult	→	WaveformViewer.tsx	outputPrediction timing trace data
9	AnalysisResult	→	Conditional Renderer	Quality score + issue count for Golden RTL gate

9. Component Reference Map

This section provides a complete reference of all frontend components and backend modules in the PulseRTL implementation, with their roles and dependencies.

File / Component	Type	Engineering Role
analysis.functions.ts	Backend Module	Gemini API fetch, JSON parsing, AnalysisResult schema
index.tsx	Route / Page	Verifier workspace — split-panel layout, pipeline trigger
synthesizer.tsx	Route / Page	HLS translation workspace — code converter
AgentPipeline.tsx	Component	8-stage animated signoff pipeline with framer-motion
ScoreCards.tsx	Component	4 engineering metric cards with progress gauges
OutputTabs.tsx	Component	8-tab results panel with download controls
WaveformViewer.tsx	Component	SVG timing diagram renderer — bus detection, hexagon labels
docs.tsx	Route / Page	Engineering documentation — pipeline and API reference
about.tsx	Route / Page	Platform overview and architecture description

10. Verification & Testing Plan

10.1 Automated / Browser Testing

- Run npm run dev and validate the full application flow locally
- Inspect split-panel responsiveness across viewport widths
- Monitor browser console and network tab for correct request/response structure
- Verify clean error state when GEMINI_API_KEY is absent from .env

10.2 Manual Verification Checklist

1	Load Verilog example — confirm pipeline executes all 8 stages in sequence
2	Load SystemVerilog example — confirm language-aware editor highlighting
3	Verify 4 metric scorecards render with correct values post-analysis
4	Download generated .sv testbench file — verify content integrity
5	Download optimized RTL file — verify synthesizable output
6	Test Golden RTL state with high-quality input (≥85% quality, 0 issues)
7	Validate waveform diagram renders with correct signal count and bus labels
8	Test HLS translation — convert C code, click 'Verify Converted Code', confirm auto-load
9	Verify LocalStorage handoff between /synthesizer and / routes
10	Confirm dark/light mode toggle using standard CSS tokens

10.3 Items Outside Current Scope

The following are explicitly excluded from the current implementation scope:

- User authentication and account management
- Persistent database storage of analysis results
- Real Verilog compilation or simulation execution
- Production-grade synthesis verification
- Multi-user collaboration or shared workspaces

11. Engineering Terminology Reference

This section defines the domain-specific terminology used throughout PulseRTL's UI, API, and documentation.

Term	Definition
RTL	Register Transfer Level — hardware description abstraction layer specifying logic operations and data transfers between registers
Verification Pipeline	Ordered sequence of automated analysis stages applied to RTL source to detect issues and generate engineering artifacts
Latch Inference	Unintended generation of a transparent latch in synthesized logic due to incomplete combinational sensitivity lists or missing else clauses
Setup/Hold Hazard	Timing violation where data does not meet setup or hold time requirements relative to the capturing clock edge
Combinational Loop	Feedback path with no register element — prevents synthesis and causes undefined simulation behavior
Synthesis-Ready	RTL code that can be passed to a logic synthesis tool without structural or syntax violations
Testbench	Verification module that provides stimulus to a design under test (DUT) and checks output responses
HLS	High-Level Synthesis — translation from algorithmic or behavioral descriptions to register-transfer-level HDL
AnalysisResult	Typed TypeScript schema returned by analyzeCode() representing the complete verification artifact set
Verification Coverage	Metric estimating the proportion of the design's functional state space exercised by the generated testbench
Timing Stability	Composite score reflecting the absence of setup/hold violations, latch inferences, and asynchronous path risks
Golden RTL State	Platform-specific term for a verified, synthesis-ready design with quality $\geq 85\%$ and zero detected issues